

CAN VISION-LANGUAGE MODELS LEARN TO EXECUTE CLASSICAL SEARCH ALGORITHMS?

Anonymous authors

Paper under double-blind review

ABSTRACT

We study whether vision-language models can learn to execute declared classical search algorithms rather than directly generate complete plans. We formulate a Search Process Policy that emits Typed Search Operations from text, visual, or multimodal state observations. A Trusted Search Runtime applies contract-valid operations, maintains bounded external Search Memory, checks algorithm-specific invariants, and charges invalid operations to the episode budget without silent repair. This separation enables model-owned exploration decisions while retaining deterministic control over search bookkeeping. Our evaluation is designed to distinguish local operational competence from full-episode structural/process competence across BFS, IW, and A*. It reports budgeted episode success separately from operation validity, invariant compliance, and invalid-operation charges. Matched text-state, visual-state, and multimodal-state conditions preserve task semantics and the search-memory interface, allowing modality effects to be assessed independently of task content. The study compares a base model, operational-only supervision, offline process supervision, and interactive data collection. Comparative experiments are ongoing; this temporary abstract will be updated with governed results.

1 INTRODUCTION

Classical search algorithms such as breadth-first search (BFS), iterative widening (IW), and A* are deterministic procedures whose correctness can be checked at every step. A system that proposes a complete plan directly, or that predicts a single next transition, is not necessarily executing such a procedure. A model can be locally correct on applicability, progression, validation, and transition/action tasks while failing to maintain the exploration and bookkeeping decisions that make a full episode a genuine execution of the declared algorithm. We call the former operational competence and the latter full-episode structural/process competence. Operational competence does not establish structural/process competence, and it does not establish executable search.

This gap is visible across the ways language and vision-language models are currently connected to planning. Classical planning and search [TODO: cite] operate through an external, deterministic reasoner; the model neither emits nor owns the search operations. Neural algorithm execution [TODO: cite] trains networks to mimic a computation, but typically on fixed-size, single-modality inputs and without an explicit, runtime-checked operation contract. LLM and VLM planning work [TODO: cite] frequently asks a model to propose a plan or a next action from a prompt, with search structure left implicit or simulated inside the context window. Runtime-constrained execution [TODO: cite] attaches deterministic tooling to a model, but usually verifies an outcome rather than checking each typed operation against algorithm invariants. The common weakness is that the model’s contribution to exploration is neither typed nor mechanically checked, so local competence and full-episode algorithm execution are conflated.

We propose a different division of responsibility. The research question is whether a vision-language model can learn to execute a declared classical search algorithm by emitting Typed Search Operations, including the model-owned decisions that determine exploration, under a Trusted Search Runtime. A Typed Search Operation is a discrete, runtime-checkable step such as expand, generate successor, update frontier, or goal test. The runtime is a deterministic executor that applies these operations, maintains algorithm state, and checks Algorithm Invariants, so the model never

owns unbounded search bookkeeping. Search Memory is an external runtime/data boundary holding frontier, visited or best-depth, novelty, and landmark state; it is not internal model state. The runtime operates without silent repair: an operation that fails the declared contract or an algorithm invariant is invalid, is charged to the episode budget, and is not silently corrected. This no-repair rule is what makes validity a mechanical judgment rather than a preference: validity is not by itself episode success, and an invalid operation always consumes budget.

The policy is a VLM trained to produce these operations rather than to propose a complete plan. The declared algorithm controls which operations are legal and which invariants must hold; one algorithm-conditioned policy is intended to serve across the named algorithms. Because the runtime holds the search bookkeeping, we can measure structural/process competence directly: over a full budgeted episode, does the sequence of operations the policy emits produce the exploration the declared algorithm specifies, and do those operations satisfy its invariants? These questions are separate from whether an individual emitted operation is locally plausible, and the evaluation contract keeps them separate. Training and evaluation are prospective. We do not claim a frozen protocol: the arm-level data sources, Search-Trace Segment construction, per-algorithm allocation, episode-budget unit, split procedure, estimands, and uncertainty rules are not yet set, and until they are the evaluation is an unfrozen plan rather than a registered protocol.

We also plan to study how the observation modality interacts with this execution. The three modality conditions are text-state, visual-state, and multimodal-state, which are intended to represent the same state and goal under matched content, semantic parity, and a fixed search-memory interface and capacity. Because no multimodal corpus and no completed modality matrix exist, these conditions are prospective controls: we intend that they preserve semantic parity and a fixed memory boundary so that any modality effect can be assessed independently of task content, but they are not yet implemented facts. The same prospective status applies to the planned BFS, IW, and A* corpora and to the training regimes we intend to compare.

This paper contributes three items. These are formulation and protocol contributions, not efficacy results.

- **The Search Process Policy formulation.** A VLM is defined to execute a declared search algorithm by emitting Typed Search Operations under a no-repair Trusted Search Runtime, with bounded external Search Memory and runtime-checked Algorithm Invariants, separating model-owned exploration decisions from deterministic search bookkeeping.
- **The no-repair runtime and the planned evaluation contract.** The evaluation is specified to report full-episode budgeted success separately from operation validity, Algorithm Invariant compliance, and invalid-operation budget charges, so that a policy cannot be credited for search it does not control and cannot hide its own errors.
- **A receipt-first evidence discipline after gates run.** Results are reported only after the governing gates run, with each stop classified explicitly; we do not treat local competence, corpus availability, or release qualification as a scientific outcome, and we do not present pending results as findings.

Current demonstrated evidence is limited to the bounded BFS-only, text-state v6 corpus and its release-qualification infrastructure. That infrastructure is corpus and release-qualification evidence; it is not a trained policy and it is not an efficacy finding. No retainable governed v6 efficacy outcome and no final comparative efficacy finding is available in the evidence base. Accordingly, this manuscript is a result-ready skeleton: the Introduction and the remainder of the paper state the question, the intervention, the planned controls, and the reporting contract, and they mark every result as pending until the corresponding gates run. We deliberately avoid claiming any trained artifact until a retainable governed outcome exists.

The remainder of the paper reviews the relevant literature, specifies the Search Process Policy and its runtime contract, describes the planned staged evaluation, and states the receipt-first reporting plan under which any empirical results will be presented.

REFERENCES

A APPENDIX

The Appendix will retain the complete evidence contract, including operation schemas, Algorithm Invariant definitions, runtime pseudocode, data and split procedures, receipt tables, statistics, adapters, and narrowly scoped feasibility details of retained infrastructure.